Assignment 1

Object Orientation

Spring 2019

1 Learning Objectives

After having completed this assignment, you should be able to

- Write, compile and execute a Java program in the NetBeans IDE;
- Define classes with attributes and methods;
- Distinguish static and dynamic methods;
- Define objects as instances of such a class;
- Use Java arrays;
- Use rudimentary Java input/output;
- Define and use a `toString()` method for user-defined classes;
- Concentrate all input/output in a single class.

2 Programming Environment

We advise you to use Java 8 and NetBeans 10 for this course. We will make use of specific features of this version of Java occasionally. NetBeans can be downloaded from `netbeans.org` for all usual platforms. Java can be downloaded from `java.com`. The standard edition Java SE is sufficient for this course.

For some of the exercises, we will provide pre-written code snippets for you to use. We always use said version of NetBeans to test our pre-written code, but there is no specific reason for it not to work in different programming environments with Java 8. Some of the computers in the Spinoza building have the Java IDE Eclipse installed. Be aware that they use the outdated Java 7 which might lead to errors when running some of our provided code.

You will be using NetBeans during the exam. We encourage you to use it during the course, even if you are currently used to a different IDE.

3 Javadoc

Every program you write for this course should contain a minimal amount of javadoc documentation: The name and student number of all authors in every file. This means that **every file** in your project has to start with a comment like the following.

```
/**
 * @author Lucy Liddels (s42)
 * @author Bob Biggens (s101)
 */
```

4 Assignment

In this assignment you have to design and implement a Java program that allows the user to create a group of students and offers the possibility to change the name of these students.

For this assignment we assume that every student has a given name and a family name, both consisting of exactly one word. Each student also has a student number, stored in an integer. These values are specified upon the construction of the student object. The student number is fixed, but the name can be changed by an appropriate setter method. The student with name Lucy Liddels and number 42 should be printed as `Lucy Liddels, s42`.

The behavior of the program to create and manipulate a group of students is summarized as follows.

1. Your program should ask the user how big the group of students is supposed to be and create a new group of the given size.

2. For each group member, your program should request a name and a student number and add this student to the group. You can choose to let the user input everything in one line and make your program parse the input, or to let the user input everything in a separate line.

   ```
   Please enter the group size:
   2
   Please enter a student:
   42 Luucy Liddels
   ```

3. After the group has been created, the program should print all the students from the group.

   ```
   The group now contains:
   Luucy Liddels, s42
   Bob Biggens, s101
   ```

4. In a loop, the program should then ask the user to input the student number, only the numerical part without the leading `s`, alongside with a new name. Every time a name is changed, the entire group should be printed again

   ```
   Student number and new given/family name?
   42 Lucy Liddels
   The group now contains:
   Lucy Liddels, s42
   Bob Biggens, s101
   ```

5. The program terminates as soon as the user enters a negative student number.

   ```
   Student number and new given/family name?
   -1
   Bye!
   ```

4.1 The Main Class

Java allows to put a `main` function in every class, which means one project can have many `main` functions. When running a program, you have to tell Java which class to use as the main class. This can become confusing when other people read your code. You should have one class in your project whose sole responsibility it is to have the `main` function. This class should have a clear name, like `Main`, or `Assignment01Main`. This class should only have the `main` function, and maybe a couple of helper functions if needed. No other class should have a `main` function.

5 Hand In

These instructions are important. Read them carefully.

5.1 Enrol In a Group

Please work in teams of two, but teams of one are also okay. Before you can hand in your project, you and your teammate have to enrol in a group in Brightspace. There are two group categories, AI and CS. Teams from AI should enrol in an AI group, everyone else in a CS group. For mixed AI/CS teams, randomly pick one. **Find an empty group and make sure you and your teammate are enrolled in the same group.** If you find other people accidentally enrolled in your group, leave the group and pick another one.

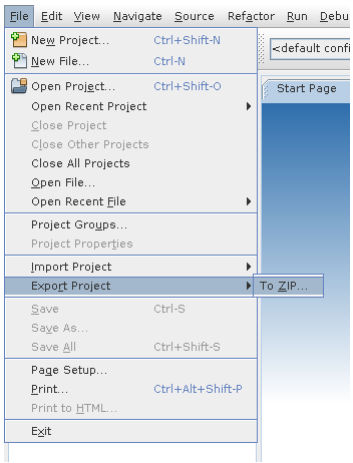
Groups will be made fixed on the day of the assignment deadline. You can only switch teams after that by sending an email to Markus Klinik.

5.2 Submit Your Project

To submit your project, follow these steps.

1. Make sure to put javadoc with your name, the name of your teammate and your student numbers into **every file**, as described in section 3.

2. Use the **project export** feature of NetBeans to create a zip file of your entire project: File → Export Project → To ZIP.



3. **Submit this zip file on Brightspace**. Do not submit individual Java files. Only one person in your group has to submit it. Submit your project before the deadline, which can be found on Brightspace.